Data Analysis Programming
Semester 2026-I
Workshop No. 2 — Technical Architecture and Bronze/Silver Pipeline

Eng. Carlos Andrés Sierra, M.Sc.
Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Welcome to the second workshop of the *Data Analysis Programming* course!

In **Workshop #1**, you defined your project angle within the *Public Opinion & Sentiment Analysis* topic, identified and validated two open data sources (one API and one web scraping target), established the extraction strategy, and formalized the project through goals, functional users, and user stories. Now, you will move into the technical implementation phase, bringing your pipeline to life using industry-standard data engineering tools.

This workshop focuses on setting up the complete technical infrastructure, designing the data preprocessing pipeline, and producing the first functional versions of both the Bronze ingestion DAG and the Silver processing DAG using Apache Airflow, all running locally inside a Docker environment.

General Workshop Definition: Technical architecture and pipeline implementation involve translating a conceptual data flow design into a working, containerized system. In this workshop, teams will configure the full project stack using Docker Compose, implement Airflow DAGs for automated data ingestion into the Bronze layer, define a comprehensive preprocessing pipeline for the Silver layer, and trigger Silver processing automatically using Airflow's FileSensor upon new Bronze file arrivals.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavirguezs@udistrital.edu.co*.

Workshop Scope and Objectives

- **Technical Architecture Definition:** Produce a detailed technical architecture document describing each component of the pipeline, tools used at each layer, and their interactions.
- **Preprocessing Pipeline Design:** Define and document all data cleaning and transformation steps required to move data from Bronze (raw JSON) to Silver (processed Parquet), including null handling, imputation strategies, outlier detection, and text normalization.
- **Docker Stack Setup:** Configure and launch the full project stack using Docker Compose, including Apache Airflow and all required dependencies.
- **Bronze Ingestion DAG:** Implement a first functional Airflow DAG that extracts data from both sources (API and web scraping) and persists raw JSON files in the `datalake_bronze/` folder using datetime-based file nomenclature.
- **Silver Processing DAG:** Implement a first functional Airflow DAG triggered by a FileSensor that detects new files in `datalake_bronze/`, applies the preprocessing pipeline, and persists processed data as Parquet files in `datalake_silver/` with timestamp-based nomenclature.

Technical Architecture and Pipeline Methodology

1. **Technical Architecture Document:**
   - Produce a detailed architecture diagram showing all system components: data sources, Airflow scheduler, Bronze layer, Silver layer, Gold layer (planned), and dashboard (planned)
   - Describe each component: purpose, technology used, inputs, outputs, and interactions with adjacent components
   - Document the Docker Compose service definitions: which containers are required (Airflow webserver, scheduler, worker, PostgreSQL metadata DB, and any additional services)
   - Justify all technology choices with clear rationale aligned with course topics (e.g., why Pandas or Polars for Silver processing, why Parquet for Silver persistence)
   - Define file nomenclature conventions for both Bronze (`source_YYYYMMDD_HHMMSS.json`) and Silver (`topic_YYYYMMDD_HHMMSS.parquet`) layers
2. **Preprocessing Pipeline Design:**
   - *Null Handling:* Identify fields that may contain null or missing values and define a strategy for each (drop, impute with mean/median/mode, fill with sentinel value, or flag)

- *Data Type Normalization:* Define correct data types for each field (e.g., timestamps as `datetime`, numeric scores as `float`, categorical fields as `category`)
- *Text Preprocessing for NLP:* Define the text cleaning pipeline: lowercasing, punctuation removal, stop word removal, tokenization, and any language-specific normalization
- *Outlier Detection and Handling:* Define statistical strategies (IQR, Z-score) for detecting and handling outliers in numeric fields (e.g., engagement counts, ratings)
- *Deduplication:* Define criteria for identifying and removing duplicate records across ingestion batches
- *Schema Enforcement:* Define the target schema for Silver Parquet files: field names, data types, and required fields
- Document all decisions with justification based on the data samples collected in Workshop #1

3. **Docker Compose Stack Setup:**

- Configure a `docker-compose.yml` file that launches a fully functional Apache Airflow environment locally
- Include all required Airflow services: webserver, scheduler, triggerer, and metadata database (PostgreSQL)
- Mount local project folders (`datalake_bronze/`, `datalake_silver/`, `datalake_gold/`, `airflow/dags/`) as Docker volumes to enable persistent data and live DAG reloading
- Define a `requirements.txt` or custom Docker image with all Python dependencies (requests, BeautifulSoup4, Pandas, Polars, PyArrow, etc.)
- Document the setup process and verify that Airflow UI is accessible at `localhost:8080`

4. **Bronze Ingestion DAG Implementation:**

- Implement an Airflow DAG using the **TaskFlow API** (`@dag` and `@task` decorators) for the Bronze ingestion pipeline
- Define tasks for: API data extraction, web scraping extraction, data validation (basic schema check), and JSON file persistence
- Configure the DAG schedule according to the ingestion periodicity defined in Workshop #1 (e.g., `@daily`, `@hourly`, or a cron expression)
- Persist each ingestion run as a separate JSON file in `datalake_bronze/` following the datetime nomenclature convention
- Include basic error handling and logging for each task
- Verify the DAG runs successfully end-to-end and produces JSON files in `datalake_bronze/`

5. **Silver Processing DAG with FileSensor:**
    - Implement a second Airflow DAG that uses a `FileSensor` operator to detect the arrival of new JSON files in `datalake_bronze/`
    - Upon detection, trigger the preprocessing pipeline defined in the architecture document
    - Apply all cleaning, normalization, outlier handling, and text preprocessing steps using Pandas or Polars
    - Persist the processed output as a Parquet file in `datalake_silver/` with a timestamp-based filename
    - Ensure that each Silver Parquet file is traceable to the Bronze source file(s) it was derived from (e.g., via metadata fields or filename convention)
    - Verify the DAG runs successfully and produces valid Parquet files readable with Pandas or PySpark

Deliverables and Documentation

1. **Technical Architecture and Pipeline Report (PDF):**
    - *Executive Summary:* Overview of the technical approach, stack decisions, and pipeline design
    - *Architecture Diagram:* Full system diagram with all components, data flows, and layer boundaries clearly labeled
    - *Preprocessing Pipeline Definition:* Complete documentation of all cleaning and transformation steps with justification
    - *Docker Stack Description:* Services, volumes, networking, and setup instructions
    - *DAG Documentation:* Description of each DAG, its tasks, schedule, trigger mechanism, and expected outputs
    - *Results Evidence:* Screenshots of Airflow UI showing successful DAG runs, sample Bronze JSON files, and sample Silver Parquet content
    - *Challenges and Decisions:* Discussion of implementation challenges encountered and how they were resolved
2. **Implementation Code:**
    - `docker-compose.yml` with all service definitions and volume mounts
    - Bronze ingestion DAG file (`airflow/dags/bronze_ingestion_dag.py`)
    - Silver processing DAG file (`airflow/dags/silver_processing_dag.py`)
    - Any supporting Python modules (API client, scraper, preprocessing functions)
    - `requirements.txt` with all Python dependencies
3. **Pipeline Output Samples:**

- At least 3 Bronze JSON files produced by the ingestion DAG across different runs
- At least 3 Silver Parquet files produced by the processing DAG
- A Jupyter Notebook in `notebooks/` demonstrating how to read and explore a Silver Parquet file using Pandas

4. **Repository Update:**
   - Add a `workshop_2/` folder containing the report PDF and any additional documentation
   - Update the repository `README.md` to reflect the current pipeline status, Docker setup instructions, and how to run the DAGs
   - Ensure all code is committed with meaningful commit messages reflecting the development process

Submission Requirements

- Submit your complete Technical Architecture and Pipeline Report as a single PDF through the designated course platform
- Include your GitHub repository link with all implementation code, Docker configuration, and pipeline output samples
- All documentation must be in **English** and follow professional technical writing standards
- Ensure proper citations for tools, frameworks, and methodologies referenced in the document
- Both DAGs must be functional and demonstrable — include evidence of successful runs in the report

Important Guidelines and Considerations

- **TaskFlow API:** Use Apache Airflow's modern `@dag` and `@task` decorator approach (TaskFlow API) for all DAG implementations. This is the current industry standard and produces cleaner, more readable pipeline code.
- **FileSensor Strategy:** The FileSensor in the Silver DAG should be configured to watch the `datalake_bronze/` directory. Understand the difference between `poke_interval` and `timeout` settings and configure them appropriately for your ingestion periodicity.
- **Parquet Nomenclature:** Each Silver Parquet file must include the timestamp of the processing run in its filename. This is critical for the Gold layer in Workshop #3, where multiple Parquet files from different time ranges will be loaded simultaneously with PySpark.
- **Idempotency:** Design your DAGs to be idempotent — running the same DAG twice for the same time period should not produce duplicate or inconsistent data.
- **Docker Volumes:** Ensure that data produced inside Docker containers is persisted to the host filesystem via volume mounts. Without this, all pipeline outputs will be lost when containers restart.
- **Preprocessing Justification:** Every preprocessing decision must be grounded in the data samples from Workshop #1. Do not define preprocessing steps in the abstract — base them on actual observed data issues.
- **Incremental Progress:** It is acceptable if the preprocessing pipeline in this workshop is a first version. Improvements based on experimentation will be incorporated in Workshop #3.

Good luck, and enjoy the process of seeing your pipeline come to life — there is a unique satisfaction in watching an Airflow DAG run end-to-end and produce real data files from sources you chose and designed yourself!